

```
import tensorflow as tf

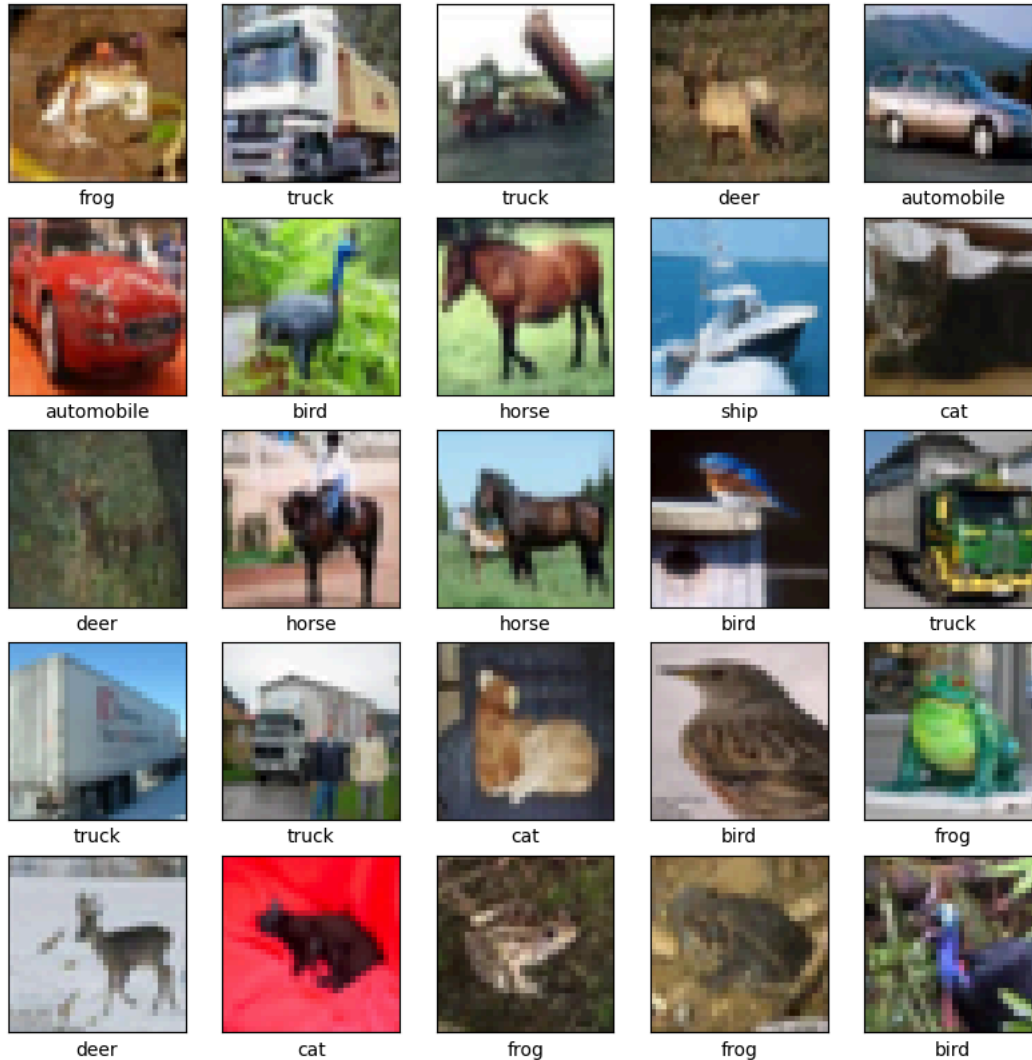
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

```
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()

# Normalize pixel values to be between 0 and 1
train_images, test_images = train_images / 255.0, test_images / 255.0
```

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i])
    # The CIFAR labels happen to be arrays,
    # which is why you need the extra index
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```



```
model = models.Sequential([
    layers.Input(shape=(32, 32, 3)),
    layers.Conv2D(32, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu')
])
```

```
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_4 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_5 (Conv2D)	(None, 4, 4, 64)	36,928

Total params: 56,320 (220.00 KB)

Trainable params: 56,320 (220.00 KB)

Non-trainable params: 0 (0.00 B)

```
model.add(layers.Flatten())
model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))
```

model.summary()

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d_2 (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_4 (Conv2D)	(None, 13, 13, 64)	18,496
max_pooling2d_3 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_5 (Conv2D)	(None, 4, 4, 64)	36,928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65,600
dense_1 (Dense)	(None, 10)	650

Total params: 122,570 (478.79 KB)

Trainable params: 122,570 (478.79 KB)

Non-trainable params: 0 (0.00 B)

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```

```
history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
```

WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use WSL2 or

Epoch 1/10
1563/1563 ————— 14s 6ms/step - accuracy: 0.4372 - loss: 1.5436 - val_accuracy: 0.5356 - val_loss: 1.2686

Epoch 2/10

1563/1563 ————— 9s 6ms/step - accuracy: 0.5792 - loss: 1.1833 - val_accuracy: 0.6048 - val_loss: 1.1024

Epoch 3/10

1563/1563 ————— 9s 6ms/step - accuracy: 0.6357 - loss: 1.0307 - val_accuracy: 0.6475 - val_loss: 1.0127

Epoch 4/10

```

1563/1563 ————— 9s 6ms/step - accuracy: 0.6686 - loss: 0.9403 - val_accuracy: 0.6575 - val_loss: 0.9871
Epoch 5/10
1563/1563 ————— 9s 6ms/step - accuracy: 0.6936 - loss: 0.8750 - val_accuracy: 0.6709 - val_loss: 0.9385
Epoch 6/10
1563/1563 ————— 9s 6ms/step - accuracy: 0.7120 - loss: 0.8188 - val_accuracy: 0.6779 - val_loss: 0.9324
Epoch 7/10
1563/1563 ————— 9s 6ms/step - accuracy: 0.7275 - loss: 0.7679 - val_accuracy: 0.6786 - val_loss: 0.9392
Epoch 8/10
1563/1563 ————— 9s 6ms/step - accuracy: 0.7464 - loss: 0.7244 - val_accuracy: 0.6903 - val_loss: 0.9044
Epoch 9/10
1563/1563 ————— 9s 6ms/step - accuracy: 0.7609 - loss: 0.6817 - val_accuracy: 0.7016 - val_loss: 0.8981
Epoch 10/10
1563/1563 ————— 9s 6ms/step - accuracy: 0.7720 - loss: 0.6457 - val_accuracy: 0.7028 - val_loss: 0.8936

```

```

plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')

test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)

```

```
313/313 - 1s - 2ms/step - accuracy: 0.7028 - loss: 0.8936
```

